

1. Print the boundary nodes of a binary tree.

```
2.  /*
3.  class Node {
4.      int data;
5.      Node left, right;
6.
7.      public Node(int d) {
8.          data = d;
9.          left = right = null;
10.     }
11. }
12. */
13.
14. class Solution {
15.     void left(Node root, ArrayList<Integer> ans){
16.         if(root==null) return;
17.         if(root.left!=null){
18.             ans.add(root.data);
19.             left(root.left,ans);
20.         }
21.         else if(root.right!=null){
22.             ans.add(root.data);
23.             left(root.right,ans);
24.         }
25.
26.     }
27.     void right(Node root, ArrayList<Integer> ans){
28.         if(root==null) return;
29.         if(root.right!=null){
30.             right(root.right,ans);
31.             ans.add(root.data);
32.         }
33.         else if(root.left!=null){
34.             right(root.left,ans);
35.             ans.add(root.data);
36.         }
37.
38.     }
39.     void leaf(Node root, ArrayList<Integer> ans){
40.         if(root==null) return;
41.         if(root.left==null&&root.right==null)ans.add(root.data);
42.         leaf(root.left,ans);
43.         leaf(root.right,ans);
44.     }
45.
46.     ArrayList<Integer> boundaryTraversal(Node root) {
47.         ArrayList<Integer> ans = new ArrayList<>();
48.         ans.add(root.data);
49.         left(root.left,ans);
50.         leaf(root.left,ans);
51.         leaf(root.right,ans);
```

```

52.         right(root.right,ans);
53.         return ans;
54.
55.     }
56. }

```

2. Print the left view of a binary tree.

```

/*
class Node {
    int data;
    Node left, right;

    Node(int val) {
        this.data = val;
        this.left = null;
        this.right = null;
    }
}
*/

class Solution {
    public ArrayList<Integer> leftView(Node root) {
        ArrayList<ArrayList<Integer>> ans = new ArrayList<>();
        Queue<Node> q = new LinkedList<>();
        q.add(root);
        while(!q.isEmpty()){
            int size = q.size();
            ArrayList<Integer> al = new ArrayList<>();
            for(int i=0;i<size;i++){
                Node cur = q.poll();
                if(cur!=null){
                    if(cur.left!=null) q.add(cur.left);
                    if(cur.right!=null) q.add(cur.right);
                }
                al.add(cur.data);
            }
            ans.add(al);
        }
        ArrayList<Integer> al = new ArrayList<>();
        for(int i=0;i<ans.size();i++){
            al.add(ans.get(i).get(0));
        }
        return al;
    }
}

```

3. Print the right view of a binary tree.

```

/**

```

```

* Definition for a binary tree node.
* public class TreeNode {
*     int val;
*     TreeNode left;
*     TreeNode right;
*     TreeNode() {}
*     TreeNode(int val) { this.val = val; }
*     TreeNode(int val, TreeNode left, TreeNode right) {
*         this.val = val;
*         this.left = left;
*         this.right = right;
*     }
* }
*/
class Solution {
    public List<Integer> rightSideView(TreeNode root) {
        List<List<Integer>> ans = new ArrayList<>();
        Queue<TreeNode> q = new LinkedList<>();
        q.add(root);
        while(!q.isEmpty()){
            int n = q.size();
            List<Integer> al = new ArrayList<>();
            for(int i=0;i<n;i++){
                TreeNode curr = q.poll();
                if(curr!=null){
                    if(curr.left!=null)q.add(curr.left);
                    if(curr.right!=null)q.add(curr.right);
                    al.add(curr.val);
                }
            }
            if (!al.isEmpty()) {
                ans.add(al);
            }
        }
        List<Integer> al = new ArrayList<>();
        for(int i=0;i<ans.size();i++){
            al.add(ans.get(i).get(ans.get(i).size()-1));
        }
        return al;
    }
}

```

4. Find the lowest common ancestor (LCA) of two nodes in a binary tree.

```

/**
* Definition for a binary tree node.
* public class TreeNode {
*     int val;
*     TreeNode left;
*     TreeNode right;

```

```

*   TreeNode(int x) { val = x; }
* }
*/
class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        if(root==null) return null;
        if(root==p||root==q)
            return root;
        TreeNode left = lowestCommonAncestor(root.left,p,q);
        TreeNode right = lowestCommonAncestor(root.right,p,q);
        if(left !=null && right !=null) return root;
        if(left==null) return right;
        return left;
    }
}

```

5. Find the path from the root to a given node in a binary tree.

```

class Solution {

    public boolean findPath(TreeNode root, int target, List<Integer> path) {

        if (root == null) {
            return false;
        }

        path.add(root.val);

        if (root.val == target) {
            return true;
        }

        if (findPath(root.left, target, path) ||
            findPath(root.right, target, path)) {
            return true;
        }

        path.remove(path.size() - 1);

        return false;
    }
}

```

6. Construct a binary tree from inorder and preorder traversal.

```

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;

```

```

*     TreeNode right;
*     TreeNode() {}
*     TreeNode(int val) { this.val = val; }
*     TreeNode(int val, TreeNode left, TreeNode right) {
*         this.val = val;
*         this.left = left;
*         this.right = right;
*     }
* }
*/
class Solution {
    int i = 0;
    public TreeNode buildTree(int[] preorder, int[] inorder) {
        HashMap<Integer,Integer> inMap = new HashMap<>();
        for(int i=0;i<inorder.length;i++)
            inMap.put(inorder[i],i);
        return constructTree(preorder,inorder,inMap,0,inorder.length-1);
    }
    TreeNode constructTree(int[] pre,int[] in,HashMap<Integer,Integer> inMap,int
start,int end){
        if(start>end)
            return null;
        TreeNode nn = new TreeNode(pre[i]);
        int idx = inMap.get(pre[i]);
        i++;
        nn.left = constructTree(pre,in,inMap,start,idx-1);
        nn.right = constructTree(pre,in,inMap,idx+1,end);
        return nn;
    }
}

```